

Quantum Routing via Beam Search

We solved QSITE's computational challenge via a beam search algorithm.

benchmark	depth	swaps	score
ghz_star	9	2	6.5
chain_trotter	9	0	4.5
ladder_trotter	7	3	6.5
qaoa_random	11	6	11.5
dense_random	23	23	34.5
vqe_layers	6	0	3.0
total			66.5

Overview

The core of our solution is a beam search algorithm, which looks something like:

```
def beam_search(program, graph):
    states = [State()]
    for i, gate in enumerate(program):
        upcoming = program[i + 1:]
        candidates = proactive_swap(run_gate(states, gate), upcoming)
        states = keep_best(candidates, upcoming)
    return states[0]

def keep_best(states, upcoming, distance, width=2048, ahead=20, alpha=0.5):
    cheapest = {layout(st): st for st in sorted(states, key=score)}
    def rank(state):
        est = sum(distances[a][b] - 1 for a, b in upcoming[:ahead]) * alpha
        return score(program) + est
    return heapq.nsmallest(width, cheapest.values(), key=rank)
```

That is, we maintain a list of current states, starting with the empty circuit. Then for each gate we have to route,

1. Build a new candidate list: states where we just perform Q2s, plus states where we proactively SWAP qubits if reused later.
2. Keep only the best candidates for the next iteration (here, the best 2048).

Then return the single best circuit at the end.

At the end, the best circuit is simply the lowest scoring one. But to know which will score lowest midway through, we need to estimate. It seems adding $0.5 * \text{sum}(\text{distance between the 20 next already placed gates})$ is a good enough estimate.¹

¹This is just a heuristic (obtained through trial and error) that happened to get us the lowest observed score. But to offer some intuition, the sum of qubit distances roughly equals the number of SWAPs we must make. Weaken that by $\alpha=0.5$ and ignore it past $\text{ahead}=20$ placements, because we shouldn't overweight our imperfect estimate. Picking the best 2048 candidates is also arbitrary - if it's too high, so many dead-end states are allowed to exist they crowd out more forward-looking moves.

Verification

For the first five challenges, we can exhaustively brute-force every potential solution to determine **our score is optimal**.

We weren't able to brute-force `dense_random` due to its extreme complexity, but searching many solutions, we found numerous ways to score 34.5 and none below. Thus it's assumed this is also (at least nearly) optimal. Specifically, attempts included

- Choosing different values for `width`, `ahead`, `alpha`
- Randomly picking which almost-duplicate state to keep, if two states have identical scores and placements.
- Taking an existing 34.5 run and mutating it